Supplementary Material S1. Computing Environment, Reproduction Record, and Median Tables

This file accompanies the manuscript "A Large Neighborhood Search for the Open Vehicle Routing Problem with Urgency-Weighted Service Latency and Tolerance-Constrained Route Selection by Tardiness and Distance". It records the software versions, seeds, budgets, and host configuration of the formal comparison, and it reports the medians and interquartile ranges that the manuscript summarizes in Section 6.2.

S1.1 Interpreter and Solvers

The tables in the manuscript come from the dependency lock `requirements.txt`. The versions below are reported only when confirmed by a record, and the stages are not given one shared Python version.

The road-network preprocessing is the one stage tied to a recorded interpreter. The virtual environment that holds OSMnx 1.9.4, and that built the driving networks used for the distances in Section 6.7, uses Python 3.8.0, as recorded in its `pyvenv.cfg`. That environment does not contain OR-Tools, PyVRP, or ALNS, so it did not run the comparisons.

The interpreter that holds the pinned packages of the lock file, and that produced the Amazon rerun in Section 6.7 and the controlled comparisons, is not identified by the console record of those runs. The record gives the package versions and not the Python minor version, so none is stated here. The compiled bytecode left in the repository was produced by more than one Python version and is not used to assign a version to a run.

Component	Formal comparison, `requirements.txt`
Python	Not recorded in the run logs
NumPy	1.24.4
SciPy	1.10.1
OR-Tools	9.10.4067
PyVRP	0.6.3
ALNS	6.0.0

PyVRP is listed only as a version record. Section 5.2 of the manuscript explains why the urgency-ignoring hard time-window PyVRP-HGS is not included in Tables 9 and 11.

The implementation languages differ. The vehicle-routing search kernel of OR-Tools 9.10.4067 is C++. The method of this paper, Weighted MLP-ILS, RLS, Priority-Greedy, and native ALNS run in Python; the ALNS package provides only a Python interface. Section 5.4 of the manuscript therefore treats the budget as the search time inside one process, not as an equal number of iterations per second.

S1.2 Host Configuration

The tables were computed on one cloud host. The table below records only what the host console reports. The console does not give the physical CPU model, and this file does not supply one.

Item	Configuration
Region	Anhui
Compute	1 GPU, L40S, 48 GB memory
CPU	24 vCPUs; the physical CPU model is not given by the console
Memory	120 GB
System disk	30 GB
Data disk	50 GB base; the console also shows an expandable data disk, and the expansion used here was 0 GB
Image	Development environment, PyTorch 2.5.1, CUDA 12.4.1 (image label V2.5.1, about 18 GB, image time 2025-03-28 00:40:13)

The search timing of the formal comparison is CPU wall-clock time and does not use the GPU. The PyTorch and CUDA in the image are not used in the solve in Section 5 and do not change the values in Table 9.

S1.3 Random Seeds and Budgets

Experiment	Seeds and budget
Tolerance development scan	Seeds 20–39; the controlled-instance budget of the main experiment
Tolerance confirmation	Seeds 60–79; the same budget
24-node main experiment	20 paired seeds in each of three scenarios; 5 s
Small-scale exact reference	8, 12, and 16 nodes, 270 instances; the exact dynamic program has no time limit; the heuristic budget in the same comparison is 0.20 s
Scale experiment	24, 48, 80, and 120 nodes; 1, 5, 10, and 30 s
Amazon replay	100 routes, five seeds each, 500 paired records; 30 s

The formal protection tolerance is 0.05. It was chosen once on the development set, and the independent seeds are used only to confirm the direction of the effect. Timing starts when the search starts. Failed and timed-out records keep their original status.

S1.4 Source of the Tables in the Manuscript

The tables are not the output of one run, except Section 6.7 and Table 18, whose source is given below. In Tables 9 and 10 and in Supplementary Tables S2 and S3, the JWL-ALNS row, the three same-objective heuristics, and OR-Tools, native ALNS, and Nearest-Neighbor come from three separate runs, using the same evaluator, the same scenario definitions, and the same wall-clock budget. The tolerance scan, the exact reference, and the attribution were rerun after the tolerance was changed to 0.05. The search archives of the curvature, the weights, and the scale experiment were saved at tolerance 0.06; the values in Tables 15 and 16, Section 6.6, and Table A1 were re-selected from those archives at $\varepsilon = 0.05$, without rerunning the search. The permutation experiment saved no archive and was rerun at $\varepsilon = 0.05$. The table below identifies the source; it does not change the values in the manuscript. Paths are relative to the repository root.

Table 9 and Supplementary Table S2, the JWL-ALNS row; Table 10 and Supplementary Table S3, the JWL-ALNS row of each scenario; Table 12, the two JWL-guided rows.
`records/table09_12_jwl_alns.csv`, the rows at $\varepsilon = 0.05$. 20 seeds in each of three scenarios, 5 s.

Tables 9 and 10 and Supplementary Tables S2 and S3, the rows for Priority-Greedy, Weighted MLP-ILS, and RLS.
`records/table09_12_same_objective.csv`. The same scenarios and seeds as the main experiment, 5 s.

Tables 9 and 10 and Supplementary Tables S2 and S3, the rows for OR-Tools, native ALNS, and Nearest-Neighbor.
`records/table09_12_travel_time.csv`. The same scenarios and seeds as the main experiment, 5 s.

Table 7.
`records/table07_tolerance_development.csv`. Development seeds 20–39.

Table 8.
`records/table08_tolerance_confirmation.csv`, the rows at $\varepsilon = 0.05$. Confirmation seeds 60–79.

Table 15.
`records/table17_curvature.csv`, re-selected at $\varepsilon = 0.05$ from `records/table17_curvature_archive.csv`. Four curvatures, five seeds in each of three scenarios per curvature, 5 s.

The permutation in Section 6.5.
`records/section65_permutation.csv`. 60 controlled instances, 5 s, $\varepsilon = 0.05$.

Table 16.
`records/table18_weights.csv`, re-selected at $\varepsilon = 0.05$ from `records/table18_weights_archive.csv`. Four weight settings, 60 instances each, 5 s.

Table 17.
`records/table19_exact.csv`. 8, 12, and 16 nodes, 270 instances; the exact dynamic program has no time limit and the heuristic budget is 0.20 s.

Table 12, the two travel-time rows.
`records/table14_travel_time_search.csv`. 60 controlled instances, 5 s.

Section 6.6 and Table A1.
`records/section66_tableA1_scale.csv`, re-selected at $\varepsilon = 0.05$ from `records/section66_tableA1_scale_archive.csv`. Four sizes, five seeds in each of three scenarios, 1, 5, 10, and 30 s.

Section 6.7 and Table 18.
`records/section67_table20/jwl_alns_shard0.csv` to `jwl_alns_shard5.csv`, with the accepted archive in `jwl_alns_archive.csv`. 100 routes, five seeds each, 30 s, $\varepsilon = 0.05$.

The methods other than JWL-ALNS in Section 6.7 come from `records/section67_other_methods/shard0.csv` to `shard5.csv`. They do not use the protection tolerance, so they were not rerun. JWL-ALNS and its priority-only output were rerun at $\varepsilon = 0.05$ and selected by Equations (18) and (19) directly. The accepted nondominated archive of each search is saved, and no record was replaced by a fallback.

Distance is the shortest path on the OpenStreetMap driving network, downloaded through OSMnx 1.9.4 from the Overpass API and saved as one graph per metropolitan area: Boston (134,887 nodes), Seattle (121,766), Los Angeles (315,749), Austin (15,941), and Chicago (160,888). The download date is not in the build log, so it cannot be stated. Each stop was snapped to the nearest network node. Across the 100 routes the median snap distance is 47 m, the 90th percentile is 63 m, and the maximum is 156 m. For node pairs left unconnected by the driving network, the distance was filled with the great-circle distance multiplied by 1.4. This fills 635 of the 466,008 off-diagonal pairs, on five routes: `RouteID_c28cddb1` (52 pairs), `RouteID_f31379fd` (79), `RouteID_fcddcd15` (76), `RouteID_5fba6b43` (148), and `RouteID_7830df77` (280). Travel time stays the published matrix. Section 6.7 reports the distance result again with these five routes removed.

`records/table08_tolerance_confirmation.csv` provides the records for Table 8. An older confirmation file at tolerance 0.06 does not correspond to that table. An earlier Amazon file that uses a different set of routes does not correspond to Table 18.

S1.5 Repository

The source code, the controlled-instance generator, the evaluator, and the result records named in S1.4 are in the GitHub repository epsilon-protected-open-routing, release manuscript-20261022, at `https://github.com/swx441/epsilon-protected-open-routing/releases/tag/manuscript-20261022`. The Amazon Last Mile raw data, the normalized matrices derived from it, and the third-party solvers are not included. The interpreter that holds the pinned packages of S1.1, and that produced the Amazon rerun, is not identified by the run logs, which do not record its minor version. The road-network preprocessing used Python 3.8.0. The host configuration is in S1.2; the physical CPU model is not in the console record and cannot be stated.

S1.6 Median and Interquartile Range in the 24-Node Main Experiment

The instances, seeds, and budget below are the same as in Tables 9 and 10 of the manuscript, and the source of the records is given in S1.4.

Supplementary Table S2. Median and interquartile range in the 24-node main experiment

Method	J_{WL} (IQR)	Tardiness (IQR)	Distance (IQR)	Travel time (IQR)
JWL-ALNS priority-protected	0.0740 (0.0505)	350.9 (140.8)	2861 (321)	3248 (363)
Priority-Greedy	0.0793 (0.0563)	556.4 (158.7)	3760 (641)	4275 (791)
Weighted MLP-ILS	0.0826 (0.0586)	447.1 (182.4)	3703 (689)	4178 (695)
RLS	0.0824 (0.0588)	451.1 (196.4)	3636 (1027)	4117 (1171)
OR-Tools Open-SVRPTW	0.0824 (0.0122)	274.5 (182.9)	2548 (312)	2870 (347)
Native ALNS	0.0875 (0.0122)	294.4 (178.8)	2488 (136)	2815 (178)
Nearest-Neighbor	0.0877 (0.0136)	312.6 (163.8)	3011 (226)	3382 (263)

Each cell is the median, with the interquartile range in parentheses. Tardiness is in generated time units. The instances are the same 60 as in Table 9 of the manuscript.

The median of J_{WL} differs from the mean in Table 9. The gap comes from the concentrated-urgency scenario, whose J_{WL} clusters near 0.027 and pulls the mean down. With 20 instances in each scenario, the median falls in one of the other two. By the median, JWL-ALNS remained the lowest of the seven methods. The median gaps against the three same-objective heuristics were 0.0053 to 0.0087, larger than the mean gaps of 0.0045 to 0.0051; once the concentrated scenario pulls the mean down, the gap is proportionally larger. The median J_{WL} of OR-Tools was 0.0824, equal to the values reported for Weighted MLP-ILS and RLS when rounded to three decimal places. Its mean is higher because the concentrated scenario does not pull it down.

The distributions of tardiness and distance point the same way as the means. Against the three same-objective heuristics, the median tardiness of JWL-ALNS was lower by 96 to 205 generated time units and the median distance by 775 to 900. Against OR-Tools and native ALNS the direction reverses: their median tardiness was lower by 56.6 and 77.4 generated time units, respectively, and their median distance was lower by 313 and 374 distance units, respectively. The median distance of Nearest-Neighbor was 150 higher than that of JWL-ALNS, in the same direction as the higher mean distance in Table 9.

Supplementary Table S3. Median and interquartile range under three service-pressure scenarios

Scenario	Method	J_{WL} (IQR)	Tardiness (IQR)	Distance (IQR)	Travel time (IQR)
Concentrated urgency	JWL-ALNS	0.0271 (0.0030)	347.2 (105.8)	2960 (370)	3364 (444)
Concentrated urgency	Priority-Greedy	0.0285 (0.0034)	562.7 (38.5)	4072 (136)	4628 (182)
Concentrated urgency	Weighted MLP-ILS	0.0271 (0.0025)	425.4 (94.7)	3719 (422)	4195 (489)
Concentrated urgency	RLS	0.0271 (0.0025)	466.1 (89.8)	3936 (574)	4499 (721)
Concentrated urgency	OR-Tools	0.0694 (0.0273)	210.0 (15.2)	2532 (216)	2862 (214)
Concentrated urgency	Native ALNS	0.0777 (0.0287)	222.2 (23.0)	2490 (139)	2817 (185)
Routine	JWL-ALNS	0.0808 (0.0085)	296.3 (22.4)	2815 (327)	3208 (300)
Routine	Priority-Greedy	0.0857 (0.0101)	378.7 (64.5)	3392 (245)	3805 (254)
Routine	Weighted MLP-ILS	0.0869 (0.0068)	378.8 (60.9)	3534 (733)	3975 (777)
Routine	RLS	0.0872 (0.0071)	320.7 (44.3)	3060 (444)	3467 (536)
Routine	OR-Tools	0.0870 (0.0084)	274.5 (19.4)	2544 (255)	2864 (306)
Routine	Native ALNS	0.0879 (0.0064)	294.4 (28.2)	2488 (132)	2815 (141)
Tight time windows	JWL-ALNS	0.0756 (0.0045)	448.4 (31.1)	2858 (276)	3249 (295)
Tight time windows	Priority-Greedy	0.0839 (0.0075)	592.8 (138.5)	3685 (498)	4165 (583)
Tight time windows	Weighted MLP-ILS	0.0854 (0.0054)	600.9 (81.4)	3848 (758)	4356 (770)
Tight time windows	RLS	0.0843 (0.0077)	604.3 (219.8)	3761 (1048)	4352 (1230)
Tight time windows	OR-Tools	0.0834 (0.0087)	408.9 (14.4)	2554 (294)	2875 (324)
Tight time windows	Native ALNS	0.0888 (0.0069)	424.9 (23.7)	2488 (132)	2815 (141)

Each cell is the median, with the interquartile range in parentheses, over 20 instances per scenario. Tardiness is in generated time units. The median of Nearest-Neighbor over the pooled 60 instances is in Supplementary Table S2.

The medians point the same way as the means in Table 10 of the manuscript. In the routine and tight time-window scenarios, the median J_{WL} of JWL-ALNS was the lowest. Under concentrated urgency it equalled the median of Weighted MLP-ILS and RLS and was lower than that of Priority-Greedy.

The medians of tardiness and distance point the same way. OR-Tools and native ALNS had lower median tardiness and distance in all three scenarios, with the smallest gap in the routine scenario and the largest under concentrated urgency.